

Royco Dusk

ROYCO PROTOCOL • TECHNICAL SPECIFICATION • v1

Dusk extends Royco Dawn so the Junior Tranche transforms both risk and liquidity profiles. Coverage capital is deployed as a Curve Stableswap-NG liquidity position paired with Senior Tranche shares, providing instant secondary liquidity at oracle-anchored prices.

Core Problem	Recursive coverage. The Junior Tranche's NAV contains Senior Tranche shares that are themselves backed by the Junior Tranche's NAV.
Resolution	Treat Senior Tranche shares held by the Junior Tranche as transiently exited; price them at pool fraction only, exclude from active claims.
Substrate	Curve Stableswap-NG via a Royco-managed factory (admin-implementation branch, CrossCurve precedent).
Asset Layout	Different assets per tranche. The Senior Tranche asset is the underlying; the Junior Tranche asset is the LP token.

01 What Dusk Does

Royco Dawn splits any priceable asset into two profiles — a Senior Tranche (capital-protected) and a Junior Tranche (first-loss). That solved risk transformation. But credit-worthiness depends on two dimensions: risk profile and liquidity profile. Dawn alone does not address the second.

Dusk extends the Junior Tranche to do both. Instead of holding plain stablecoins as coverage, the Junior Tranche deploys its capital into a Curve Stableswap-NG liquidity position paired against Senior Tranche shares:

- **Risk transformation.** Coverage still flows from the Junior Tranche to the Senior Tranche exactly as in Dawn.
- **Liquidity transformation.** The Junior Tranche's LP position gives Senior Tranche holders an instant secondary market with continuous quoted prices.

All quantities in this document are in NAV units (the kernel's native value denomination, e.g. USD) unless otherwise noted.

02 The Recursive Coverage Problem

Putting Senior Tranche shares into the Junior Tranche's LP creates a circular dependency. All Senior Tranche shares are protected pro-rata by the Junior Tranche's NAV. But if that NAV is partly composed of Senior Tranche shares, those shares are themselves protected by the NAV that contains them.

Dusk eliminates this by treating Senior Tranche shares inside the Junior Tranche's LP as *transiently exited*. They are part of the Junior Tranche's asset side, not active senior claims, and should not be covered.

03 AMM Substrate

Three properties drive the substrate choice:

- **Fungible LP tokens.** The Junior Tranche's pool ownership is a clean ERC-20 balance. This rules out Uniswap v3/v4, where positions are NFTs.
- **Bonding-curve execution bounded by composition.** Stableswap satisfies this natively.
- **Per-asset rate oracle.** Stableswap-NG's asset type 1 lets the pool consult an external oracle, anchoring quotes to the kernel's per-share NAV.

Pool Configuration

Senior Tranche share	→	asset type 1 (rate-oracled)
Rate oracle	→	Royco tranche oracle
Stablecoin side	→	asset type 0 (standard ERC-20)
Amplification	→	tuned per market

Amplification A is tuned per market; the Curve range for stablecoin-like pairs is 200–2000. Higher values keep execution closer to the oracle rate over a wider composition range and raise the soft-guarantee threshold derived later.

Deployment uses a Royco-managed Stableswap-NG factory pointing at the *admin-implementation* pool blueprint (Curve-maintained, used in production by CrossCurve). The factory constructor sets `fee_receiver` and `admin` to Royco-controlled addresses, routing admin fees to Royco, placing A ramping and per-market fee tuning under Royco control, and inheriting Curve's audit posture on the pool implementation. Pool LP tokens are eligible for Curve gauges via standard DAO vote — CrossCurve's pools using this blueprint received gauge controller approval in late 2024, establishing direct precedent. Once approved, CRV emissions accrue to the Junior Tranche through the gauge as additional yield.

04 Kernel Asset Layout

Dusk is the first non-identical-asset Dawn kernel: the Senior Tranche asset $\neq$ the Junior Tranche asset.

```

SENIOR_TRANCHE_ASSET    = underlying yield-bearing asset (e.g. sUSDe)
stOwnedYieldBearingAssets raw underlying held by the kernel

JUNIOR_TRANCHE_ASSET    = Curve Stableswap-NG LP token
jtOwnedYieldBearingAssets LP tokens held by the kernel

```

All Senior Tranche asset lives in the kernel, not in the pool. The pool holds stablecoins and Senior Tranche shares; the shares are claims that resolve through the Senior Tranche's redemption path.

05 The Mechanism

Effective Supply

The kernel tracks one new state variable: `internalShares` — the count of Senior Tranche shares the Junior Tranche's LP holds, as of the last accounting sync. These shares are treated as transiently exited: they do not accrue coverage and do not share in the Senior Tranche's effective NAV.

The denominator for deposits and redemptions becomes:

```
effectiveSupply = totalSupply - internalShares
```

External LPs — the only ones with active claims — redeem at the Senior Tranche's effective NAV divided by effective supply. Internal shares cannot claim on the Senior Tranche's effective NAV (that would be recursive). The Junior Tranche realises value from those shares only when the LP unwinds.

Quoter

The quoter computes each tranche's raw NAV against the cached partition, ensuring it is non-recursive and idempotent within a sync. The Senior Tranche's raw NAV is the per-share underlying value scaled to the external share count; the Junior Tranche's raw NAV is the underlying value of internal shares plus the pool's stablecoin balance:

```

ST_RAW_NAV = assetPrice × stAssets × ( stShares - internalShares ) / stShares

JT_RAW_NAV = assetPrice × stAssets × internalShares / stShares
             + stablePrice × stables

```

Here `assetPrice` is the per-unit NAV price of the underlying, `stAssets` is the kernel-held underlying (`stOwnedYieldBearingAssets`), `stShares` is the total Senior Tranche supply, `stables` is the pool's stablecoin balance, and `stablePrice` is typically 1. Their sum equals the total system value: **conservation holds by construction**.

```
ST_RAW_NAV + JT_RAW_NAV = assetPrice × stAssets + stablePrice × stables
```

06 Sync Flow

Between any two accounting syncs, two independent things happen: a *composition shift* (swaps moved Senior Tranche shares between external holders and the pool) and *real economics* (the underlying accrued yield or depreciated; the LP collected fees or experienced slippage). The recomposition updates checkpoints to reflect the new partition; the PnL waterfall then marks the recomposed checkpoints to current prices.

Step 1: Recomposition

Read the current LP composition to obtain `newInternalShares`. Define the scaling factor as the ratio of new external share count to old:

```
scalingFactor = ( stShares - newInternalShares ) / ( stShares - internalShares )
```

Senior Tranche checkpoints (scaled)

```
ST_RAW_NAV      = LAST_ST_RAW_NAV      × scalingFactor
ST_EFFECTIVE_NAV = LAST_ST_EFFECTIVE_NAV × scalingFactor
ST_IL           = LAST_ST_IL           × scalingFactor
```

Junior Tranche checkpoints

```
JT_RAW_NAV      = LAST_JT_RAW_NAV
JT_EFFECTIVE_NAV = ST_RAW_NAV + JT_RAW_NAV - ST_EFFECTIVE_NAV
JT_IL           = LAST_JT_IL           × scalingFactor
```

After writing the recomposed checkpoints, set `internalShares = newInternalShares`.

Step 2: PnL Waterfall

Run Dawn's existing accounting waterfall with the quoter's current-price raw NAVs against the recomposed checkpoints. The waterfall processes deltas in this order:

1. Junior Tranche delta
 - Loss: JT absorbs; excess spills to Senior Tranche IL
 - Gain: Recover Senior Tranche IL first; JT accrues residual
2. Senior Tranche delta
 - Loss: JT covers via `min( loss, JT_EFFECTIVE_NAV )`
 - Gain: Recover ST IL, then JT IL, then distribute yield via YDM

This ordering ensures the Junior Tranche's delta (including any coverage-premium cost from composition shifts) is fully processed before the coverage step runs.

07 Derivation of the Recomposition

Core Principle

The recomposition has two distinct jobs. First, remove *phantoms* from the deltas the waterfall will see — changes in raw NAV caused by share reclassification with no corresponding economic event. Real economic events (cash flows, price changes) must remain in the deltas. Second, preserve the Dawn invariant that links impermanent-loss accumulators to the cross-tranche claim (JT_IL matches the cross-claim ST_EFF – ST_RAW, and ST_IL scales with external share count). Both jobs are necessary; together they fully account for the partition shift.

Why the Senior Tranche Has a Phantom

Without recomposition, at unchanged prices with shares entering the pool:

$$\text{deltaST} = - (\text{assetPrice} \times \text{stAssets} \times \text{deltaInternal} / \text{stShares}) < 0$$

The waterfall would interpret this as a Senior Tranche loss and trigger Junior Tranche coverage. But nothing economic happened — shares were relabeled with no offsetting cash flow on the Senior Tranche side. This is a pure phantom, removed by scaling. `deltaInternal` here is `newInternalShares – internalShares`.

Why the Junior Tranche Has No Phantom in Raw NAV

The same reclassification adds share backing to the Junior Tranche's pool. But unlike the Senior Tranche side, every reclassification on the Junior Tranche side is paired with a real swap: stablecoins left the pool to pay for the incoming shares. At unchanged prices:

$$\text{deltaJT} = \text{deltaInternal} \times (\text{perShareRaw} - \text{swapPrice})$$

Where `perShareRaw = assetPrice × stAssets / stShares` is the raw backing per share and `swapPrice` is the average execution price the pool paid. The reclassification component (backing added) and the cash flow component (stablecoins leaving) always occur together:

- **At par** (`swapPrice = perShareRaw`): `deltaJT = 0`. Perfect cancellation.
- **Off par**: `deltaJT` equals the execution residual — a real economic gain or loss.

There is no phantom in `deltaJT`. Every reclassification on the Junior Tranche side is accompanied by a real cash flow. Therefore the Junior Tranche's raw NAV checkpoint is left unchanged. ■

Why Both Impermanent-Loss Accumulators Scale

The phantom-removal argument applies to raw NAV because `JT_RAW` is a quoter output. The impermanent-loss accumulators are different: they are state variables that track outstanding recovery claims, and the right rule for them depends on the mechanics of how external supply changes.

What JT_IL represents. `JT_IL` is JT's claim that future Senior Tranche asset appreciation should priority-flow back to JT, in compensation for past coverage. The recovery happens through the waterfall on `deltaST`, which carries `perShareRaw × externalShares × appreciation%` of the total `stAsset` gain. The recovery rate per dollar of `stAsset` appreciation is therefore proportional to external-share count.

Why Dusk scales JT_IL on recomposition. When the partition shifts via the pool, the exiting shares move *internal*, not gone. `stAssets` is unchanged. The `stAsset` appreciation that previously flowed entirely through `deltaST` now splits: a portion still flows through `deltaST` (proportional to remaining external), and the rest flows directly into `JT_RAW` as appreciation on the newly internal shares JT owns through the pool. JT captures the second portion automatically through `deltaJT`.

If `JT_IL` stayed absolute under this dual-channel structure, JT would recover the full original claim from the external channel *and* also capture the internal-channel appreciation directly: a double-count. The exiting shares' portion of the coverage debt was already settled in cash at the moment of the swap (the pool paid them out at oracle price, which embeds JT's prior coverage into the per-share value they walked away with). Scaling `JT_IL` by the external-share ratio prevents the double-count and makes JT's eventual recovery match its original claim, exactly.

What this preserves. Scaling `JT_IL` preserves the economic invariant that JT recovers exactly its original coverage debt, no more, no less. `ST_IL` scales for the analogous reason: the exiting shares took their proportional share of unrecovered loss with them via the realized oracle price at the swap, and the remaining external retains the same per-share IL.

Consequence for Junior Tranche Effective NAV

After scaling the Senior Tranche and leaving the Junior Tranche's raw NAV unchanged, conservation gives:

$$\begin{aligned} \text{JT_EFFECTIVE_NAV} &= \text{LAST_JT_EFFECTIVE_NAV} \\ &+ (\text{LAST_ST_EFFECTIVE_NAV} - \text{LAST_ST_RAW_NAV}) \\ &\times \text{deltaInternal} / \text{externalShares} \end{aligned}$$

Where `externalShares = stShares - internalShares` (the old external count). When coverage has been applied (Senior Tranche effective NAV exceeds raw NAV), the Junior Tranche's effective NAV increases. This is not phantom — it reflects fewer external shares to protect. The corresponding cost (the stablecoin premium the pool paid for those shares at the oracle rate) sits in `deltaJT` and is processed by the waterfall **before** the coverage step runs.

08 Delta Composition

The *Senior Tranche delta* captures purely the change in value of the underlying asset between syncs, scaled to the external fraction. Curve pool mechanics never touch the kernel's underlying balance, so they cannot bleed into this delta.

The *Junior Tranche delta* captures the sum of three real economic events:

- **Yield or depreciation on internal shares.** Proportional to the count of internally-held Senior Tranche shares.
- **LP-making revenue.** Curve trading fees, swap slippage, and (when coverage is active) the oracle-rate premium cost from shares entering above raw backing.

- **Bonding-curve execution residuals.** The difference between actual Stableswap execution and the oracle rate.

All three flow through Dawn’s existing waterfall unchanged.

09 Coverage Sizing: Dynamic β

In Dawn, β (BETA) is a static per-market constant representing the Junior Tranche’s sensitivity to Senior Tranche downside, scaled to WAD (10^{18} fixed-point). $\beta = 0$ means uncorrelated; $\beta = \text{WAD}$ means fully correlated.

In Dusk, the Junior Tranche’s portfolio is a shifting mix of stablecoins (uncorrelated) and internal Senior Tranche shares (fully correlated). Dusk computes β per-sync as a composition-weighted average:

$$\text{BETA} = \left(\text{assetPrice} \times \text{stAssets} \times \text{internalShares} / \text{stShares} \times \text{WAD} \right) / \text{JT_RAW_NAV}$$

BETA is bounded above by WAD, attained when the pool holds zero stablecoins. For single-asset kernels (all existing Dawn markets), this reduces to the configured static β with no behaviour change. The kernel updates the accountant’s beta parameter before each sync.

10 Coverage Invariant Under LP Swaps

This section proves that, at constant Senior Tranche asset prices, swap activity through the pool cannot worsen utilization — unconditionally for the trade direction relevant to exit liquidity, and conditionally for the opposite direction with explicit bounds.

The argument has three steps. First, the coverage numerator depends only on stAsset valuation, not on how shares are partitioned between external and internal supply (Lemma 1). Second, swap geometry produces a slippage residual whose sign is fixed by trade direction (Lemma 2). Third, every swap also collects a fee that reliably exceeds adverse residuals across well-parameterized operating ranges (Theorem).

Definitions

A Stableswap-NG pool with rate oracle has a *rate-adjusted balance point* where the stablecoin reserves equal the rate-adjusted value of the Senior Tranche share reserves. Relative to this balance point:

- A *balancing trade* moves the pool toward the balance point — for example, selling Senior Tranche shares into a pool that is overweight stablecoins. The Stableswap bonding curve pays a small premium above the oracle rate to incentivise rebalancing.
- An *imbancing trade* moves the pool away from the balance point — for example, selling Senior Tranche shares into a pool that is already overweight Senior Tranche shares. The bonding curve pays below the oracle rate, penalising further imbalance.

The exit-liquidity case Dusk is built for is consistently imbalancing: Senior Tranche holders selling shares into a pool whose share side is already accumulating from prior exits.

Lemma 1 — Partition-Invariant Coverage Numerator

Claim. The coverage numerator $ST_RAW_NAV + \beta \cdot JT_RAW_NAV$ collapses to a quantity Π that depends only on stAsset valuation, not on how shares are split between external and internal supply.

Intuition. Coverage exists to absorb stAsset depreciation. The pool's stablecoin reserves are insulated from stAsset price moves; they never need to be covered against. Whether a Senior Tranche share is held externally or sits inside the pool, it represents the same claim on stAssets and carries the same exposure. Π is therefore the total Senior Tranche-asset exposure in the system, which is invariant to the partition.

Proof. Substituting the dynamic- β definition into the coverage numerator:

$$\begin{aligned} ST_RAW_NAV + JT_RAW_NAV \times \beta / WAD \\ &= \text{perShareRaw} \times \text{externalShares} + \text{perShareRaw} \times \text{internalShares} \\ &= \text{perShareRaw} \times \text{stShares} \\ &= \text{assetPrice} \times \text{stAssets} \\ &:= \Pi \end{aligned}$$

The stablecoin term in JT_RAW_NAV cancels because β weights JT_RAW_NAV by the underlying-backed fraction of pool inventory. Π is constant between syncs, and $\text{util} = \text{COVERAGE} \times \Pi / JT_EFFECTIVE_NAV$ therefore depends only on $JT_EFFECTIVE_NAV$. ■

Lemma 2 — Sign and Magnitude of the Slippage Residual

Setup. A swap of NAV size t executes against the bonding curve. Define $\sigma(t, \text{state})$ as the difference between the pool NAV after the swap and what the pool NAV would have been if the same trade had executed at the oracle rate (no slippage). This is the slippage residual, separate from the fee.

Sign claim. $\sigma \geq 0$ for imbalancing trades and $\sigma \leq 0$ for balancing trades.

Intuition for sign. Stableswap's bonding curve is more curved away from the balance point and flatter near it. An imbalancing trade pushes into the curved region: the trader pays a premium (worse than oracle), and that premium accrues to the pool as additional NAV. A balancing trade does the reverse: the trader collects a premium for restoring balance, and that premium comes out of the pool's NAV. The geometry is unconditional — it follows from concavity of the invariant alone.

Magnitude claim (balancing case). For trades that pull the pool toward balance:

$$|\sigma(t, \text{state})| \leq K(\text{state}) \times t^2 / (A \times D)$$

Intuition for the bound. Three factors govern how costly a balancing trade is to the pool:

- **Quadratic in trade size t :** the marginal slippage rate grows linearly with how far the trade pushes along the curve, so total slippage scales as the integral, t^2 . Small trades carry negligible residuals; the cost only matters at scale.

- **Inverse in $A \cdot D$:** amplification A determines how flat the curve is around balance, and D is the pool size. $A \cdot D$ is the effective “depth” of the curve. Doubling either halves the slippage on a given trade.
- **State-dependent factor $K(\text{state})$:** a dimensionless imbalance multiplier. $K = 0$ exactly at balance (no slippage to pay out, since the curve is flat there), $K \leq 1$ across normal operating ranges, and K grows unboundedly only as one reserve nears depletion. K is what captures “how much harder is it to extract liquidity from an already-imbalanced pool.”

The bound is loose at balance and tightest at extreme imbalance. For the operating ranges Dusk targets — arbitrage keeps the pool within roughly 5% of balance — K stays well below 1.

Theorem — Utilization Non-Increasing Under Swaps

Claim. At constant Senior Tranche asset prices, every swap satisfies one of:

- **Hard guarantee:** $\sigma \geq 0$, fee adds further to $JT_EFFECTIVE_NAV$, utilization strictly decreases.
- **Soft guarantee:** $\sigma < 0$ (pool pays slippage to trader), but the fee dominates, so $JT_EFFECTIVE_NAV$ still grows and utilization decreases.
- **Bounded breach:** $\sigma < 0$ and the residual exceeds the fee. Utilization can rise, but only by an amount bounded by Lemma 2.

Proof. The change in $JT_EFFECTIVE_NAV$ from a single swap decomposes into two terms:

$$\Delta JT_EFF = lpFeeRate \times t + \sigma(t, \text{state})$$

where $lpFeeRate = (1 - adminFeeShare) \times poolFeeRate$ is the LP-retained portion of the pool fee. The recomposition (which scales ST_EFF by external supply) and the waterfall (which routes the realized σ and fee through to JT_EFF) together produce this decomposition uniformly across all market states. By Lemma 1, Π is unchanged. Since util is monotonically inverse in $JT_EFFECTIVE_NAV$:

$$util_new \leq util_old \Leftrightarrow \Delta JT_EFF \geq 0 \Leftrightarrow lpFeeRate \times t \geq |\sigma(t, \text{state})|$$

Imbalancing trades. By Lemma 2, $\sigma \geq 0$. Both terms in ΔJT_EFF are non-negative, the inequality holds unconditionally, and utilization strictly decreases by the fee alone. *Hard guarantee.*

Balancing trades. By Lemma 2, $\sigma \leq 0$ with magnitude bounded by $K \cdot t^2 / (A \cdot D)$. The fee dominates whenever:

$$lpFeeRate \times t \geq K \times t^2 / (A \times D)$$

Solving for t , the threshold below which utilization is guaranteed non-increasing:

$$t \leq lpFeeRate \times A \times D / K(\text{state})$$

For trades exceeding the threshold, the worst-case utilization increase is:

$$\Delta_{util} \leq (K \times t^2 / (A \times D) - lpFeeRate \times t) / JT_EFF$$

Bounded, predictable, and tunable through pool parameter selection. ■

Numerical instantiation

For Curve-standard parameters ($A = 1000$, $D = \$10M$ pool, $poolFeeRate = 4$ bps, $adminFeeShare = 50\%$, $K \leq 1$ in moderate imbalance):

- $lpFeeRate = 2$ bps
- Hard-guarantee threshold for balancing trades = \$2M per swap, or 20% of pool size.
- Setting $adminFeeShare = 0$ doubles $lpFeeRate$ to 4 bps, doubling the threshold to 40% of pool size.
- Raising A to 2000 doubles $A \cdot D$, doubling the threshold again to 80% of pool size.

With aggressive parameter selection, the hard guarantee covers essentially all realistic single-block trades. Imbalancing trades — the dominant flow during exit pressure — are unconditionally safe regardless of parameters.

11 Junior Tranche Deposits

Junior Tranche deposits are dual-asset. The depositor contributes both *underlying* and *stables* — never LP tokens directly. The deposit composes Dawn’s existing primitives in a fixed five-call sequence dictated by the coverage check ordering. The protocol does not implement a deposit router; execution is handled by the solver network (CoW, linch, LiFi, and equivalents) that already routes capital across DeFi.

The Sequence

Every Junior Tranche deposit of X underlying and Y stables follows the same five calls, atomic in a single transaction:

- **Single-sided stable add to pool.** Y stables enter the Curve pool, returning LP tokens.
- **jtDeposit with the resulting LP.** The kernel’s `jtOwnedYieldBearingAssets` grows; `JT_EFFECTIVE_NAV` gains Y . This is what enables the next step.
- **stDeposit with X underlying.** The kernel mints X new external Senior Tranche shares to the executor. The coverage check passes because step 2 already grew `JT_EFFECTIVE_NAV`.
- **Single-sided share add to pool.** The freshly-minted shares enter the pool, returning more LP tokens.
- **jtDeposit with the new LP.** The recomposition observes the new internal share count and absorbs the partition shift cleanly.

The executor ends with $X + Y$ NAV worth of Junior Tranche shares and forwards them to the depositor.

Why No Router

Solvers already compete to route capital across DeFi optimally. They source assets through whichever DEX route is cheapest, flashloan when economical, batch with other operations, and choose deposit ratios to minimize Curve imbalance fees against the current pool state. A fixed router contract can only do one thing; a solver network does whatever is best for a given user, market state, and asset availability.

Cementing the deposit logic in a router would mean: one path fixed in code, audit overhead for the router, upgrade governance for every Curve evolution, and no competition on execution quality. Leaving it to solvers means: many competing implementations, each evolving independently, no protocol-level surface area, and best-execution by construction.

The protocol's responsibility ends at exposing the primitives. The five-step sequence is the constraint that any executor must satisfy — but the protocol neither enforces nor implements it.

Bootstrap Is the Empty-Pool Case

Bootstrap is the same five-step flow when the pool starts empty. Curve's first-deposit semantics handle the empty state: step 1's single-sided contribution simply sets $D = Y$, no bonding-curve geometry is consulted. From step 2 onwards, the flow is identical to any subsequent deposit. The pool is balanced at the moment any external party can interact with it; the empty-side singularity is never reachable from outside the bootstrap transaction.

Why Every Deposit Follows This Pattern

Four properties make this the right flow for every Junior Tranche deposit, not just bootstrap:

- **Pool composition tracks exit pressure exclusively.** Senior Tranche deposits don't touch the pool. Standard Senior Tranche redemptions don't touch the pool. Junior Tranche deposits through this sequence and Junior Tranche redemptions through the kernel's pro-rata unwrap both move the pool proportionally. The only flow that shifts pool composition is Senior Tranche holders selling shares into the pool — exactly Dusk's exit-liquidity use case. Pool imbalance becomes a clean signal of unmet exit demand.
- **Rebalancing is incentivized for free.** The depositor's chosen (X, Y) ratio determines whether the deposit is balancing, neutral, or imbalancing relative to current pool composition. Curve's existing imbalance fee structure charges more for imbalancing deposits and less for balancing ones. Depositors maximizing Junior Tranche shares per dollar contributed prefer to bring the scarcer asset, naturally pushing the pool back toward balance. No protocol-level rebalance logic is required.
- **Coverage scaling is automatic.** When the depositor brings X underlying, the kernel mints exactly X new Senior Tranche shares and absorbs them into the pool as internal supply. $JT_EFFECTIVE_NAV$ grows by $X + Y$. Capital scales coherently across all three sides — Senior Tranche supply, Junior Tranche capacity, and pool inventory — without any side getting ahead.
- **External Senior Tranche capacity is preserved.** A Junior Tranche deposit never reduces headroom for new external Senior Tranche deposits. Step 3 mints X external Senior Tranche shares; step 4 absorbs exactly those X back into pool inventory as internal supply. Net change to external share count is zero and per-share raw NAV is preserved at par, so ST_RAW_NAV is invariant through the flow. $JT_EFFECTIVE_NAV$ grows by $X + Y$. The cap on new external Senior Tranche deposits therefore

strictly expands.

Cost Structure

The five-step sequence pays Curve's imbalance fee on each single-sided leg (steps 1 and 4). The fee splits between an LP-fee portion (stays in the pool, recouped via the Junior Tranche's LP holdings) and an admin-fee portion (goes to Royco). Net cost to the depositor is the admin-fee portion only — typically a few basis points of total deposit value at standard Curve fee parameters. For markets where this overhead matters, setting `adminFeeShare = 0` recovers the full LP fee; protocol revenue then routes through the kernel's yield-distribution mechanism instead.

Solvers optimize this cost further by, for example, sourcing the underlying from the pool itself (a *stables_in* swap followed by Senior Tranche redemption) when cheaper than an external DEX, or flashloaning to skip a leg. None of this requires protocol changes.

12 Junior Tranche Redemption

Redemption delivers underlying assets (stablecoins and Senior Tranche asset) without disturbing external Senior Tranche holders' per-share effective value. Dawn's claim decomposition splits the total NAV into four components:

```
stClaimOnJTRawNAV    = max( 0, ST_EFFECTIVE_NAV - ST_RAW_NAV )
jtClaimOnSTRawNAV    = max( 0, JT_EFFECTIVE_NAV - JT_RAW_NAV )
stClaimOnSelfRawNAV  = ST_RAW_NAV - jtClaimOnSTRawNAV
jtClaimOnSelfRawNAV  = JT_RAW_NAV - stClaimOnJTRawNAV
```

For a Junior Tranche redeemer claiming a fraction f of the tranche (`redeemerShares / jtSupply`), the total NAV delivered is:

```
NAV_to_redeemer = f × ( jtClaimOnSelfRawNAV + jtClaimOnSTRawNAV ) = f × JT_EFFECTIVE_NAV
```

These components are settled differently:

- **jtClaimOnSelfRawNAV portion.** Delivered as Junior Tranche asset (LP tokens, unwrapped to stablecoins + Senior Tranche shares).
- **jtClaimOnSTRawNAV portion.** Delivered as Senior Tranche asset directly from the kernel's underlying balance (Dawn's existing pattern, unchanged in Dusk).

Decomposition by Market State

Conservation of value ($ST_EFF + JT_EFF = ST_RAW + JT_RAW$) implies that at most one of the cross-tranche claims is positive at any time.

- **Healthy with yield accrued** ($JT_EFF > JT_RAW, ST_EFF = ST_RAW$): $jtClaimOnSelfRawNAV = JT_RAW, jtClaimOnSTRawNAV = JT_EFF - JT_RAW$.

- **Coverage applied** ($ST_EFF > ST_RAW, JT_EFF < JT_RAW$): $jtClaimOnSelfRawNAV = JT_EFF$, $jtClaimOnSTRawNAV = 0$.

LP Unwrap Mechanism

The kernel unwraps a NAV-weighted fraction of its LP holdings, not a flat pro-rata of redeemer shares:

$$lpToWithdraw = f \times (jtClaimOnSelfRawNAV / JT_RAW_NAV) \times jtOwnedYieldBearingAssets$$

In the healthy state, $jtClaimOnSelfRawNAV = JT_RAW_NAV$, so $lpToWithdraw$ collapses to $f \times jtOwnedYieldBearingAssets$ — flat pro-rata. In the coverage state, $jtClaimOnSelfRawNAV = JT_EFF < JT_RAW_NAV$, so $lpToWithdraw$ is reduced by the ratio JT_EFF / JT_RAW . The unredeemed fraction stays in the pool to honour future Senior Tranche redemption claims.

Three-step execution

- **Pro-rata pool withdrawal.** Remove $lpToWithdraw$ LP tokens from the pool. The pool returns proportional stablecoins and Senior Tranche shares.
- **Par-coupled burn.** Burn the Senior Tranche shares received and reduce the kernel's underlying balance by $burned \times stAssets / stShares$. $stShares$ decreases by the burned amount.
- **Transfer.** Deliver stablecoins from step 1, Senior Tranche asset from step 2, plus any $jtClaimOnSTRawNAV$ portion drawn directly from the kernel's underlying balance.

Preservation Proofs

External share count

The par-coupled burn removes the same count from both supplies:

$$\begin{aligned} externalShares_new \\ &= (stShares - burned) - (internalShares - burned) \\ &= externalShares \quad \blacksquare \end{aligned}$$

Per-share Senior Tranche effective NAV

ST_EFF is unchanged by JT redemption (the kernel only debits JT_EFF by the redemption amount). External share count is preserved (above). Therefore the per-external-share effective NAV — the rate at which external Senior Tranche holders can redeem — is preserved exactly:

$$ST_EFF_new / externalShares_new = ST_EFF / externalShares \quad \blacksquare$$

Per-share ST_RAW is exactly preserved when $jtClaimOnSTRawNAV = 0$ (no draw beyond the par-coupled portion). When $jtClaimOnSTRawNAV > 0$, the additional draw reduces $stAssets$ without burning shares, so per-share ST_RAW decreases slightly. The redemption rate that matters to external Senior Tranche holders is per-share ST_EFF , which is preserved in both cases.

NAV conservation

The redeemer's LP fraction times the pool's value yields the `jtClaimOnSelfRawNAV` portion; the `jtClaimOnSTRawNAV` portion is added directly. Substituting `lpToWithdraw / lpSupply = f × jtClaimOnSelfRawNAV / JT_RAW_NAV`:

```
NAV_to_redeemer = ( f × jtClaimOnSelfRawNAV / JT_RAW_NAV )
                  × ( stablePrice × stables + assetPrice × stAssets × internalShares / stShares )
                  + f × jtClaimOnSTRawNAV
                  = f × jtClaimOnSelfRawNAV + f × jtClaimOnSTRawNAV
                  = f × JT_EFFECTIVE_NAV ■
```

(Using `JT_RAW_NAV = stablePrice × stables + assetPrice × stAssets × internalShares / stShares` from the quoter formula.)

13 Senior Tranche Redemption Override

Dawn's default kernel routes `stClaimOnJTRawNAV` (the Senior Tranche's claim on Junior Tranche raw NAV during coverage) through Junior Tranche asset units. In Dusk this is inappropriate: a redeeming Senior Tranche holder should receive Senior Tranche asset, not LP tokens.

The Dusk kernel overrides the Senior Tranche redemption path:

- `stClaimOnSelfRawNAV` **portion**. Senior Tranche asset from the kernel's underlying balance, with par-coupled accounting.
- `stClaimOnJTRawNAV` **portion (coverage settlement)**. Also delivered as Senior Tranche asset. Sourced by partial LP unwrap: the kernel unwraps enough of the Junior Tranche's LP to cover this NAV, burns any Senior Tranche shares received (par-coupled), and pays the redeemer in Senior Tranche asset.

This preserves Dawn's coverage semantics (Senior Tranche holders are made whole using Junior Tranche assets) while respecting Dusk's asset structure (Senior Tranche holders never receive LP tokens).

14 Open Items

Effective-external-supply boundary cases. When `externalShares = 0` (post-bootstrap, before any external Senior Tranche deposits) or approaches zero, the rate oracle's denominator approaches zero. The kernel needs an explicit fallback: oracle rate equals `lastSTEffectiveNAV / externalShares` when `externalShares > 0`, otherwise `perShareRaw`. The fallback is consistent: in any healthy state, `perShareRaw` equals the oracle rate.

Curve admin fee allocation. Setting `adminFeeShare > 0` routes a portion of every Curve fee to Royco rather than the pool's LP (the Junior Tranche). This trades Junior Tranche yield and coverage strength for protocol revenue. Each market should choose this tradeoff explicitly; the default of zero is reasonable for new markets prioritizing Junior Tranche capacity, with protocol revenue routed instead through the kernel's yield-distribution mechanism.

Pool parameter selection. Amplification A , pool fee rate, and admin fee share jointly determine the soft-guarantee threshold ($\text{lpFeeRate} \times A \times D / K$) and the rebalancing-incentive strength on Junior Tranche deposits. Higher A and lower admin fee both raise the threshold. Each market should choose parameters such that the threshold covers any single-block adversarial trade size for that market's expected liquidity.

Yield-distribution treatment for LP revenue. The yield distribution module was designed for Senior Tranche-originated yield. Whether the same yield-share split is appropriate for Junior Tranche-originated LP-making revenue (Curve trading fees collected by the Junior Tranche's LP) should be confirmed.

MEV and sandwich resistance. Standard Curve sandwich risk applies. The pool provides liquidity availability, not guaranteed execution at the oracle rate for arbitrary trade sizes.

Senior Tranche redemption gas cost. The override requires partial LP unwrap when coverage has been applied. Gas profile and slippage tolerance configuration should be reviewed against typical exit trade sizes.

Self-liquidation bonus interaction. Dawn's self-liquidation bonus on Senior Tranche redemption uses Junior Tranche NAV. In Dusk, this requires unwrapping LP to source the bonus in Senior Tranche asset; the override path must handle this case explicitly.

15 Summary

Dusk turns the Junior Tranche from a pure coverage vehicle into a coverage-and-liquidity vehicle without changing Dawn's core promises. The single new state variable, the recomposition, the effective-supply share math, and the composition-weighted dynamic β break the recursive coverage problem and deliver continuous secondary liquidity for any Senior Tranche on any priceable asset.

The recomposition has two distinct jobs grounded in different invariants. Raw NAV checkpoints follow a structural asymmetry: Senior Tranche reclassification creates a phantom that must be scaled away, while Junior Tranche reclassification is paired with a real swap and leaves no phantom in raw NAV. Impermanent-loss accumulators scale by the external-share ratio because the partition shift creates a dual recovery channel: external Senior Tranche supply still flows through the waterfall, while the newly internal supply flows directly into JT through pool ownership. Scaling preserves the Dawn invariant that JT recovers exactly its original coverage debt without double-counting across both channels.

Junior Tranche deposits are dual-asset by design. Every deposit composes the same five-call sequence over Dawn's existing primitives, executed atomically by the solver network rather than a protocol-level router. Pool composition therefore shifts only when Senior Tranche holders exit through the secondary path, making pool imbalance a clean signal of exit demand. Curve's natural imbalance fee structure incentivizes rebalancing deposits without protocol intervention. Bootstrap is the empty-pool special case of the same flow, with no first-swap singularity exposed externally.

Junior Tranche redemption uses a NAV-weighted LP unwrap that collapses to flat pro-rata in the healthy state and reduces unwrap volume by $\text{JT_EFF} / \text{JT_RAW}$ in the coverage state, preserving external Senior Tranche holders' per-share effective NAV. Senior Tranche redemption is routed through a Dusk-specific override that always settles in Senior Tranche asset, never in LP tokens.

Deployed on Curve Stableswap-NG with rate oracles tied to Royco's tranche oracle and Royco as pool admin, Dusk captures exchange-tier fee economics from day one. The coverage invariant is approximately preserved under LP swaps: imbalancing trades guarantee non-increasing utilization unconditionally, and balancing trades stay within a bounded residual whose magnitude is determined by the chosen pool amplification.

Royco Dusk Technical Specification — v1. Subject to refinement during audit.

Properties verified against the royco-dawn codebase. Not financial advice.